Apache Airflow in Docker (Windows) with an External Postgres Database

Complete setup guide for running Airflow via Docker Desktop on Windows, connected to a **separately installed** Postgres server (not the Postgres container bundled with Airflow's Compose file).

Part 1 — Install Required Software

Before touching Airflow, install these three tools on your Windows machine.

1. Visual Studio Code

Used to edit your DAG files and config files (.env, postgresql.conf, pg_hba.conf).

1. Download from: https://code.visualstudio.com/
2. Run the installer. During setup, check these boxes for convenience:
    - "Add to PATH"
    - "Register Code as an editor for supported file types"
    - "Add 'Open with Code' action" (both for files and folders)
3. Launch VS Code once to confirm it opens correctly.
4. (Optional but helpful) Install the **Python** extension from the Extensions tab (Ctrl+Shift+X → search "Python" → Install) for syntax highlighting in your DAG files.

2. Docker Desktop

Runs the Airflow containers.

1. Download from: https://www.docker.com/products/docker-desktop/
2. Run the installer. If prompted, enable **WSL 2** (Windows Subsystem for Linux) — Docker Desktop will guide you through this if it's not already installed.
3. Restart your PC if prompted.
4. Launch Docker Desktop and wait for the whale icon in the system tray to show it's running (no longer "starting...").
5. Increase resources if needed: **Settings → Resources** → set Memory to at least 4GB (8GB recommended if your machine can spare it).
6. Confirm it works by opening PowerShell and running:

7. docker --versiondocker compose version

Both should print version numbers without error.

3. PostgreSQL (native Windows install)

This is the external database your Airflow DAG will read/write to (separate from Airflow's own internal metadata database, which runs inside a container automatically).

1. Download the installer from: https://www.postgresql.org/download/windows/ (This uses the EnterpriseDB installer, the most common method.)
2. Run the installer:
    - Choose an install directory (default is fine)
    - Select components — keep **PostgreSQL Server**, **pgAdmin 4**, and **Command Line Tools** checked
    - Choose a data directory (default is fine)
    - **Set a password for the postgres superuser** — remember this, you'll need it for the Airflow connection later
    - Port: leave as default 5432
    - Locale: default is fine
3. Let the installer finish; it will also install **pgAdmin 4**, a GUI tool for managing your database, and add PostgreSQL as a Windows service so it starts automatically.
4. Confirm it's running: Win + R → services.msc → look for postgresql-x64-<version> with status "Running".
5. Open **pgAdmin 4** from the Start menu, connect using the password you set, and confirm you can see the default postgres database under **Servers → PostgreSQL → Databases**.

With all three installed, continue to Part 1.

Part 2 — Install Airflow via Docker Desktop

Prerequisites recap

- ✅ VS Code installed (for editing files)
- ✅ Docker Desktop installed and running
- ✅ At least 4GB RAM allocated to Docker Desktop (Settings → Resources)
- ✅ PostgreSQL installed natively on Windows, with a known postgres user password

2. Create the project folder

mkdir airflow-docker

cd airflow-docker

3. Download the official Airflow Compose file

curl -LfO 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'

4. Create required folders

mkdir dags, logs, plugins, config

5. Create the .env file

"AIRFLOW_UID=50000" | Out-File -FilePath .env -Encoding ascii

6. Initialize the Airflow database

docker compose up airflow-init

Wait for airflow-init exited with code 0.

7. Start all Airflow services

docker compose up -d

Check status:

docker compose ps

Note: the bundled postgres container in this Compose file is used only for **Airflow's own internal metadata** (DAG runs, task states, users, etc). You don't need to touch or remove it — your DAG will use a *separate* connection pointing to your external Postgres instead.

8. Open the Airflow UI

Go to http://localhost:8080

- Username: airflow
- Password: airflow

Part 3 — Configure External Postgres to Accept Docker Connections

By default, Postgres only listens on localhost and only trusts local connections. Since Airflow runs inside a container, you need to open it up.

1. Edit postgresql.conf

Location is typically:

C:\Program Files\PostgreSQL\<version>\data\postgresql.conf

Set:

listen_addresses = '*'

2. Edit pg_hba.conf

Same folder. Add (or confirm) these lines:

```
# TYPE  DATABASE      USER        ADDRESS             METHOD
local   all           all                             scram-sha-256
host    all           all         127.0.0.1/32        scram-sha-256
host    all           all         ::1/128             scram-sha-256
host    all           all         0.0.0.0/0           scram-sha-256
```

3. Restart the Postgres service

Via Services app:

- Win + R → services.msc → find postgresql-x64-<version> → Restart

Or via elevated PowerShell:

Restart-Service postgresql-x64-18

(Replace 18 with your installed version.)

4. Open the firewall port

Elevated PowerShell:

New-NetFirewallRule -DisplayName "PostgreSQL 5432" -Direction Inbound -Protocol TCP -LocalPort 5432 -Action Allow

5. Confirm the airflow database exists

In pgAdmin, run:

\l

If airflow isn't listed, create it:

CREATE DATABASE airflow;

Part 4 — Create the Airflow Connection

In the Airflow UI: **Admin → Connections → +**

Field	**Value**
Connection Id	pg_con
Connection Type	Postgres
Host	host.docker.internal ⚠️ **not** localhost
Database	airflow
Login	postgres
Password	admin
Port	5432

Why host.docker.internal and not localhost: Inside a Docker container, localhost refers to the container itself — not your Windows machine. Docker Desktop provides host.docker.internal as a special DNS name that resolves to your host machine, which is how the container reaches a Postgres server running natively on Windows.

Click **Save**.

Part 5 — Final DAG Code

Save this as:

C:\Users\yanna\airflow-docker\dags\process_employees.py

```
import datetime
import pendulum
import os
import requests
from airflow.sdk import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator



@dag(
    dag_id="process_employees",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
)
def ProcessEmployees():
    create_employees_table = SQLExecuteQueryOperator(
        task_id="create_employees_table",
        conn_id="pg_con",
        sql="""
            CREATE TABLE IF NOT EXISTS employees (
                "Serial Number" NUMERIC PRIMARY KEY,
```

```
        "Company Name" TEXT,
        "Employee Markme" TEXT,
        "Description" TEXT,
        "Leave" INTEGER
    );""",
)

create_employees_temp_table = SQLExecuteQueryOperator(
    task_id="create_employees_temp_table",
    conn_id="pg_con",
    sql="""
      DROP TABLE IF EXISTS employees_temp CASCADE;
      CREATE TABLE employees_temp (
        "Serial Number" NUMERIC PRIMARY KEY,
        "Company Name" TEXT,
        "Employee Markme" TEXT,
        "Description" TEXT,
        "Leave" INTEGER
      );""",
)

@task
def get_data():
    data_path = "/opt/airflow/dags/files/employees.csv"
    os.makedirs(os.path.dirname(data_path), exist_ok=True)
```

```
    url = "https://raw.githubusercontent.com/apache/airflow/main/airflow-core/docs/tutorial/pipeline_example.csv"

    response = requests.request("GET", url)

    with open(data_path, "w") as file:
      file.write(response.text)

    # IMPORTANT: this must run AFTER the file is closed, not inside the `with` block,
    # otherwise the CSV may not be fully flushed to disk yet.
    postgres_hook = PostgresHook(postgres_conn_id="pg_con")
    postgres_hook.copy_expert(
      sql="COPY employees_temp FROM STDIN WITH CSV HEADER DELIMITER AS ',' QUOTE '\"'",
      filename=data_path,
    )

  @task
  def merge_data():
    query = """
      INSERT INTO employees
      SELECT *
      FROM (
        SELECT DISTINCT *
        FROM employees_temp
      ) t
      ON CONFLICT ("Serial Number") DO UPDATE
```

```
        SET
          "Employee Markme" = excluded."Employee Markme",
          "Description" = excluded."Description",
          "Leave" = excluded."Leave";
    """
    try:
        postgres_hook = PostgresHook(postgres_conn_id="pg_con")
        conn = postgres_hook.get_conn()
        cur = conn.cursor()
        cur.execute(query)
        conn.commit()
        return 0
    except Exception as e:
        return 1

[create_employees_table, create_employees_temp_table] >> get_data() >> merge_data()

dag = ProcessEmployees()
```

Part 6 — Run It

1. Wait 30–60 seconds after saving the file — the scheduler auto-scans the dags folder.
2. Refresh http://localhost:8080.
3. Open process_employees → **Code** tab → confirm your latest version loaded.
4. Toggle the DAG **On**.

5. Click ▶ **Trigger**.
6. Watch the **Grid** view — all 4 tasks should turn green:
    - create_employees_table
    - create_employees_temp_table
    - get_data
    - merge_data
7. Verify in pgAdmin:
8. SELECT * FROM employees_temp;SELECT * FROM employees;

Both should now return rows.

Troubleshooting Reference

Error	Cause	Fix
connection to server on socket "/var/run/postgresql/.s.PGSQL.5432"	Host field blank/wrong	Set Host to host.docker.internal
Connection refused on 127.0.0.1 / ::1	Host field set to localhost	Change to host.docker.internal
database "airflow" does not exist	Wrong database name in connection	Set Database field to a DB that actually exists, or CREATE DATABASE airflow;
Network is unreachable on an IPv6 address	Docker resolved host.docker.internal to IPv6, which isn't routable	Usually resolves once pg_hba.conf + firewall + listen_addresses are all correctly set; Postgres will fall back to IPv4
'Cursor' object has no attribute 'copy_expert'	psycopg3 doesn't support cur.copy_expert()	Use postgres_hook.copy_expert(sql=..., filename=...) on the hook, not the cursor

Error	Cause	Fix
employees_temp created but 0 rows	copy_expert() called *inside* the with open() block, before the file is flushed	Move the copy_expert() call outside/after the with block
duplicate key value violates unique constraint "pg_type_typname_nsp_index"	Stale Postgres catalog entry from a rapid drop/create	In pgAdmin: DROP TABLE IF EXISTS employees_temp CASCADE; then clear/retry the task in Airflow
mkdir: A positional parameter cannot be found	PowerShell doesn't support mkdir -p a b c	Use mkdir a, b, c
id: The term 'id' is not recognized	No id command in PowerShell	Use "AIRFLOW_UID=50000" | Out-File -FilePath .env -Encoding ascii

---

Useful Commands

```
# Check container status
docker compose ps

# Start Airflow
docker compose up -d

# Stop Airflow
docker compose down

# View scheduler logs (DAG parse errors)
docker compose logs scheduler --tail=50

# Restart external Postgres (Windows)
Restart-Service postgresql-x64-18

# Open Postgres port in firewall (Windows, elevated PowerShell)
New-NetFirewallRule -DisplayName "PostgreSQL 5432" -Direction Inbound -Protocol TCP -LocalPort 5432 -Action Allow
```

scheduled__2026-08-18T00:00:00+00:00 ✓ Success

Logical Date	Run Type	Start Date	End Date	Duration	Dag Version(s)
2026-08-17 20:00:00	Scheduled	2026-08-17 21:57:49	2026-08-17 21:57:53	00:00:03.937	v2

Add a note...

Task Instances | Asset Events | Audit Log | Code | Details

+ Filter

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
merge_data		✓ Success	2026-08-17 21:57:52	1	@task	00:00:00.259
get_data		✓ Success	2026-08-17 21:57:50	1	@task	00:00:00.751
create_employees_temp_table		✓ Success	2026-08-17 21:57:49	1	SQLExecuteQueryOperator	00:00:00.695
create_employees_table		✓ Success	2026-08-17 21:57:49	1	SQLExecuteQueryOperator	00:00:00.659